LAPORAN PRAKTIKUM

POSTTEST 7

ALGORITMA PEMROGRAMAN DASAR



Disusun oleh:

Daniel Vivaldi Sagala (2609106054)

Kelas (B2 ‘26)

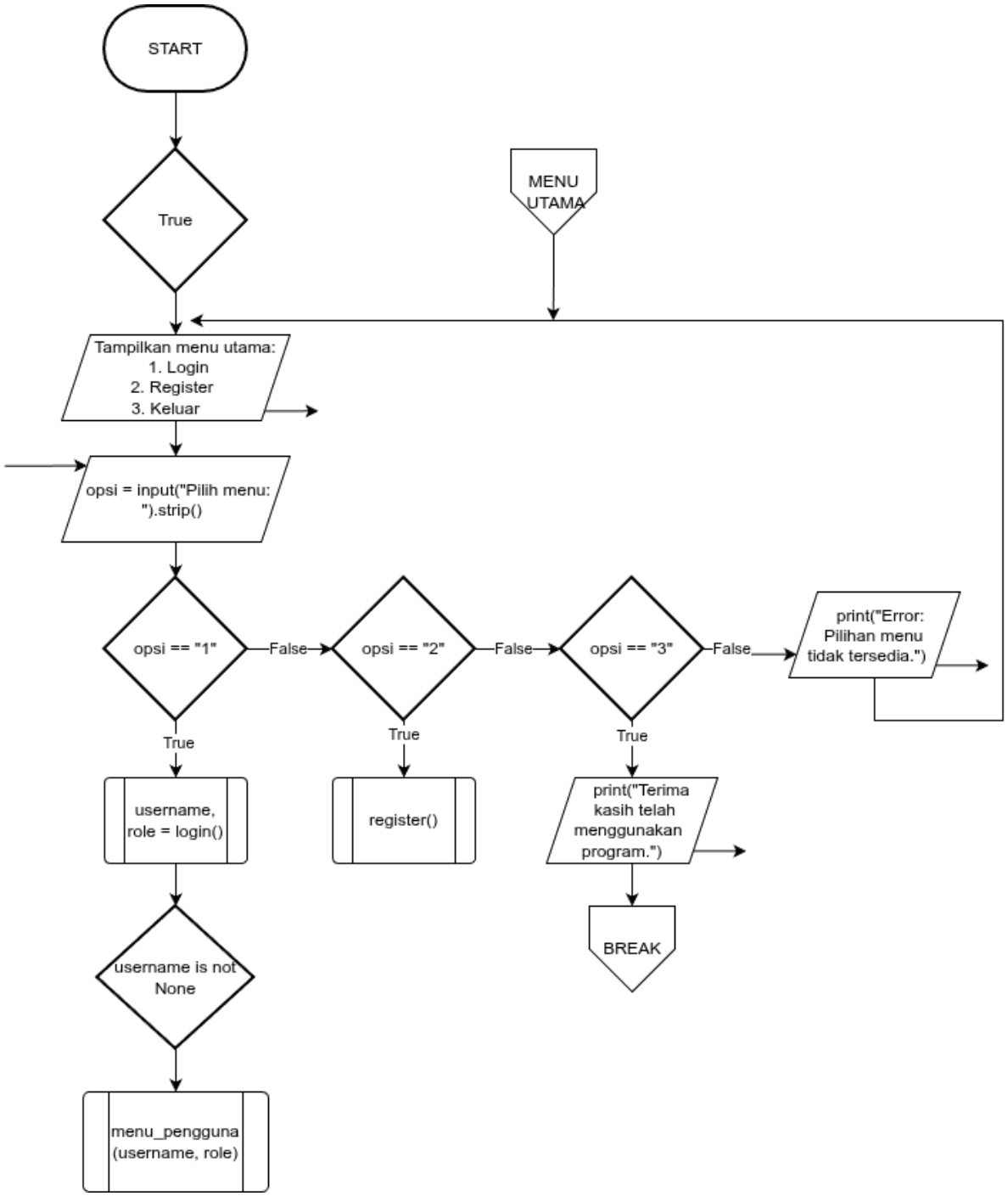
PROGRAM STUDI INFORMATIKA

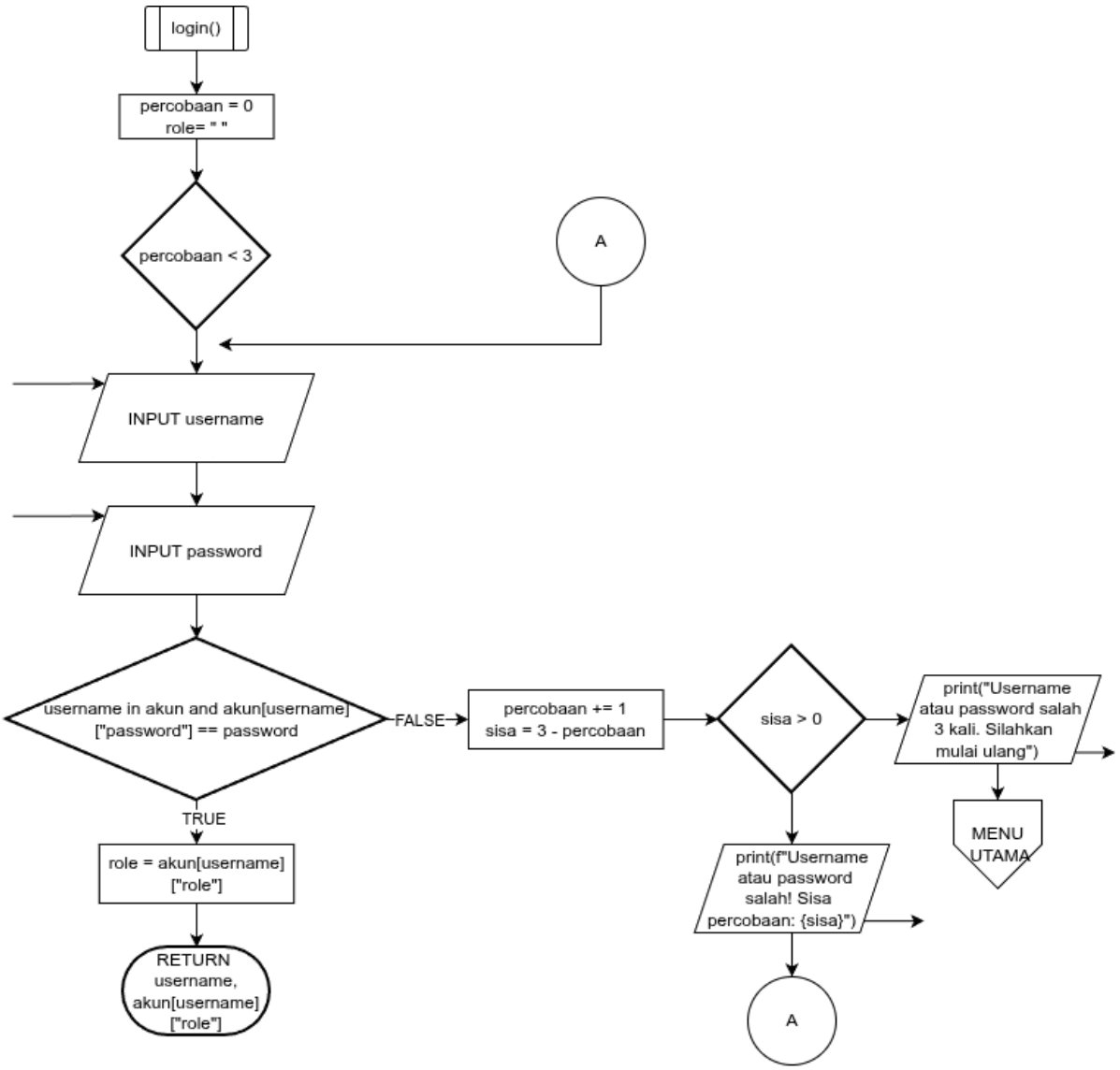
UNIVERSITAS MULAWARMAN

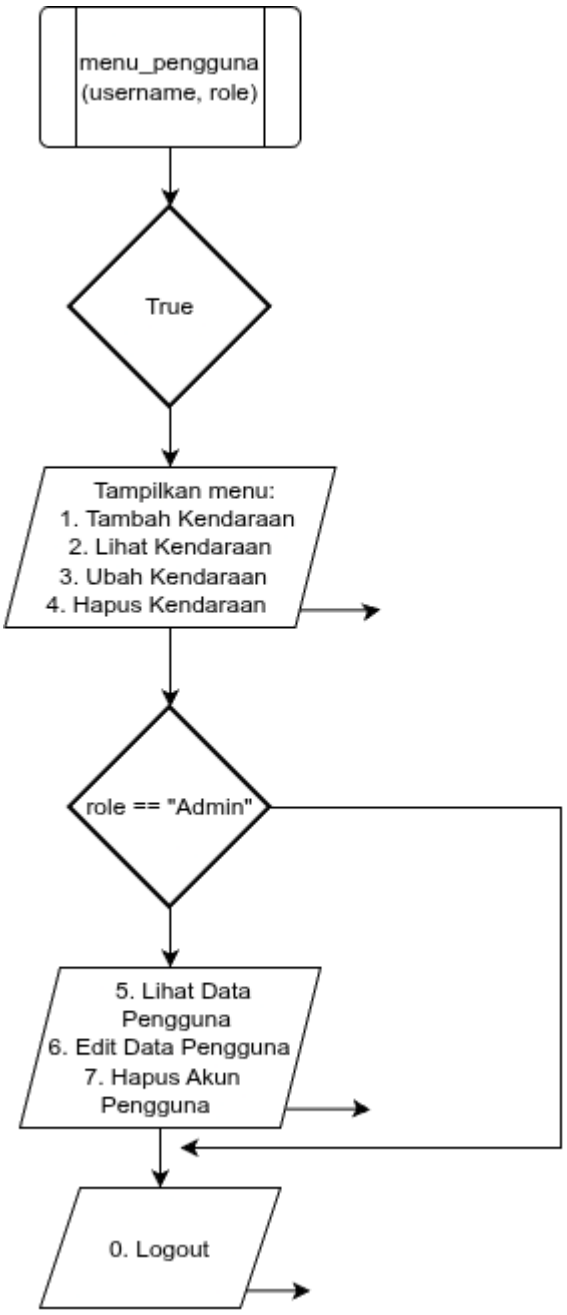
SAMARINDA

2026

1. Flowchart


START
True
MENU UTAMA
Tampilkan menu utama:
1. Login
2. Register
3. Keluar
opsi = input("Pilih menu: ").strip()
opsi == "1"
False
opsi == "2"
False
opsi == "3"
False
print("Error: Pilihan menu tidak tersedia.")
True
True
True
username, role = login()
register()
print("Terima kasih telah menggunakan program.")
BREAK
username is not None
menu_pengguna (username, role)


login()
percobaan = 0
role= " "
percobaan < 3
A
INPUT username
INPUT password
username in akun and akun[username]["password"] == password
FALSE
percobaan += 1
sisa = 3 - percobaan
sisa > 0
print("Username atau password salah 3 kali. Silahkan mulai ulang")
MENU UTAMA
TRUE
role = akun[username]["role"]
print(f"Username atau password salah! Sisa percobaan: {sisa}")
RETURN username, akun[username]["role"]
A


menu_pengguna
(username, role)
True
Tampilkan menu:
1. Tambah Kendaraan
2. Lihat Kendaraan
3. Ubah Kendaraan
4. Hapus Kendaraan
role == "Admin"
5. Lihat Data
Pengguna
6. Edit Data Pengguna
7. Hapus Akun
Pengguna
0. Logout

2. Deskripsi Singkat Program

Program ini merupakan aplikasi Sistem Pendataan Kendaraan berbasis konsol yang dibuat menggunakan bahasa pemrograman Python untuk mengelola data kendaraan, seperti nomor plat, nama pemilik, jenis kendaraan, merek, dan tahun kendaraan. Program ini menerapkan sistem CRUD (Create, Read, Update, Delete), yaitu menambahkan data kendaraan baru, menampilkan data kendaraan, mengubah data yang sudah ada, serta menghapus data kendaraan.

Program ini dilengkapi dengan fitur Multi User yang membagi pengguna menjadi dua tingkat hak akses, yaitu Admin dan User. Admin memiliki akses untuk menggunakan seluruh fitur CRUD serta mengelola data akun pengguna, sedangkan User hanya dapat mengelola data kendaraan miliknya sendiri sesuai dengan hak akses yang diberikan.

Program ini juga menerapkan Error Handling untuk menangani kesalahan input, validasi data untuk mencegah data yang tidak sesuai, pembatasan percobaan login maksimal tiga kali, pengecekan duplikasi nomor plat, serta konfirmasi sebelum penghapusan data.

Dalam pengembangannya, program menggunakan library PrettyTable untuk menampilkan data kendaraan dan pengguna dalam bentuk tabel agar lebih rapi dan mudah dibaca. Selain itu, digunakan library Datetime untuk mendapatkan tahun saat ini sehingga validasi tahun kendaraan dapat dilakukan secara otomatis dan tidak menerima tahun yang melebihi tahun berjalan.

Program dibuat dalam bentuk Modular Programing untuk memudahkan maintanance serta agar struktur program menjadi lebih tertata.

3. Source Code

A. Fitur Menampilkan Menu Utama

Menampilkan tiga pilihan utama, yaitu Login, Register, dan Keluar dari program.

Source Code:

```
def main():
    while True:
        os.system("cls" if os.name == "nt" else "clear")
        print("🏍️  SISTEM PENDATAAN KENDARAAN  🚗.")
        print("1. Login")
        print("2. Register")
        print("3. Keluar")

        opsi = input("Pilih menu: ").strip()

        if opsi == "1":
            os.system("cls" if os.name == "nt" else "clear")
            username, role = login()

            if username is not None:
                os.system("cls" if os.name == "nt" else "clear")
                menu_pengguna(username, role)

        elif opsi == "2":
            os.system("cls" if os.name == "nt" else "clear")
            register()

        elif opsi == "3":
            os.system("cls" if os.name == "nt" else "clear")
            print("Terima kasih telah menggunakan program.")
            break

        else:
            print("Error: Pilihan menu tidak tersedia.")

if __name__ == "__main__":
    main()
```

B. Fitur Login

1. Pengguna memasukkan username dan password.
2. Sistem mencocokkan data dengan daftar akun yang tersimpan.
3. Terdapat batas maksimal tiga kali percobaan login.
4. Jika login berhasil, sistem menampilkan username dan level akses pengguna.
5. Jika login gagal tiga kali, pengguna akan dikembalikan ke menu utama program.

Source Code:

```
def login():
    print("=== LOGIN ===")
    percobaan = 0

    while percobaan < 3:
        username = input("Username: ").strip()
        password = input("Password: ").strip()

        role = ""
        if username in akun and akun[username]["password"] == password:
            role = akun[username]["role"]
            return username, akun[username]["role"]
        else:
            percobaan += 1
            sisa = 3 - percobaan
            if sisa > 0:
                print(f"Username atau password salah! Sisa percobaan: {sisa}")
            else:
                print("Username atau password salah 3 kali. Silahkan mulai
ulang")
                input("\nTekan Enter untuk melanjutkan...")
    return None, None
```

C. Fitur Register

1. Pengguna baru dapat membuat akun dengan memasukkan username dan password.
2. Sistem memeriksa apakah username sudah digunakan.
3. Jika username belum terdaftar dan data tidak kosong, akun baru akan disimpan.
4. Setiap akun yang didaftarkan secara otomatis memiliki level akses User.

Source Code:

```
def register():
    try:
        print("========== REGISTER ==========")

        username = input("Buat username: ").strip()
        password = input("Buat password: ").strip()

        if not username or not password:
            raise ValueError("Username dan password wajib diisi.")

        if username in akun:
            raise ValueError("Username sudah digunakan.")

        akun[username] = {
            "password": password,
            "role": "User"
        }

        print("Registrasi berhasil. Silakan login.")
        input("\nTekan Enter untuk melanjutkan...")

    except ValueError as error:
        print("Error:", error)
```

D. Fitur Hak Akses Berdasarkan Level

Program memiliki dua level akses, yaitu Admin dan User.

1. Admin: dapat mengelola data kendaraan melalui fitur tambah, lihat, ubah, dan hapus kendaraan. Admin juga dapat melihat, mengedit, dan menghapus akun pengguna dengan ketentuan tertentu.
2. User: dapat menambahkan, melihat, mengubah, dan menghapus data kendaraan miliknya sendiri. User tidak memiliki akses ke menu pengelolaan akun pengguna.

Source Code:

```
print("1. Tambah Kendaraan")
print("2. Lihat Kendaraan")
print("3. Ubah Kendaraan")
print("4. Hapus Kendaraan")

if role == "Admin":
    print("5. Lihat Data Pengguna")
    print("6. Edit Data Pengguna")
    print("7. Hapus Akun Pengguna")

print("0. Logout")
```

E. Fitur Menu Pendataan Kendaraan (CRUD)

1. Tambah Kendaraan (Create)

Pengguna dapat menambahkan data kendaraan baru. Sistem memvalidasi nomor plat, nama pemilik, jenis kendaraan, merek, dan tahun kendaraan sebelum menyimpan data.

Source Code:

```
def tambah_kendaraan(username):
    try:
        print("======= TAMBAH KENDARAAN =======")

        plat = input("Nomor plat: ").strip().upper()

        if plat == "":
            raise ValueError("Nomor plat wajib diisi.")

        for data in data_kendaraan.values():
            if data["plat"] == plat:
                raise ValueError("Nomor plat sudah terdaftar.")

        pemilik = input("Nama pemilik: ").strip()
        jenis = input("Jenis kendaraan (Mobil/Motor): ").strip().capitalize()
        merek = input("Merek kendaraan: ").strip()
        tahun_input = input("Tahun kendaraan: ").strip()

        if pemilik == "" or merek == "":
            raise ValueError("Data tidak boleh kosong.")
        elif jenis not in ("Mobil", "Motor"):
            raise ValueError("Jenis harus Mobil atau Motor.")
        elif not validasi_tahun(tahun_input):
            raise ValueError("Tahun kendaraan tidak valid.")
        else:
            id_baru = len(data_kendaraan) + 1
            data_kendaraan[id_baru] = {
                "plat": plat,
                "pemilik": pemilik,
                "jenis": jenis,
                "merek": merek,
                "tahun": int(tahun_input),
                "username": username
            }

            print("Data kendaraan berhasil ditambahkan!")
            print("ID kendaraan:", id_baru)

    except ValueError as error:
```

```
        print("Error:", error)
```

2. Lihat Kendaraan (Read)

Menampilkan seluruh data kendaraan yang tersimpan, meliputi ID, nomor plat, nama pemilik, jenis, merek, tahun, dan pengguna yang memasukkan data.

Source Code:

```
def tampilkan_kendaraan():
    print("========== DATA KENDARAAN ==========")

    if hitung_kendaraan() == 0:
        print("Belum ada data kendaraan.")
        return

    tabel = PrettyTable()

    tabel.field_names = [
        "ID",
        "Nomor Plat",
        "Nama Pemilik",
        "Jenis",
        "Merek",
        "Tahun",
        "Input Oleh"
    ]

    for id_kendaraan, data in data_kendaraan.items():
        tabel.add_row([
            id_kendaraan,
            data["plat"],
            data["pemilik"],
            data["jenis"],
            data["merek"],
            data["tahun"],
            data["username"]
        ])

    tabel.align = "l"
    print(tabel)

    daftar_id = list(data_kendaraan.keys())
    jumlah_rekursif = hitung_kendaraan_rekursif(daftar_id)
    print("Jumlah :", jumlah_rekursif)
```

3. Ubah Kendaraan (Update)

Pengguna memasukkan ID kendaraan yang ingin diubah. Sistem memeriksa keberadaan data dan hak akses sebelum memperbarui informasi kendaraan.

Source Code:

```
def ubah_kendaraan(username, role):
    try:
        tampilkan_kendaraan()

        if not data_kendaraan:
            return

        print("\n========= UBAH KENDARAAN =========")
        id_input = input("Masukkan ID kendaraan: ").strip()

        if not id_input.isdigit():
            raise ValueError("ID harus berupa angka.")

        id_kendaraan = int(id_input)
        data = cari_kendaraan(id_kendaraan)

        if data is None:
            raise ValueError("Data tidak ditemukan.")

        if role != "Admin" and data["username"] != username:
            raise ValueError("Anda tidak memiliki akses.")

        print("Data lama:", data)

        plat = input("Nomor plat baru: ").strip().upper()

        if not plat:
            raise ValueError("Nomor plat wajib diisi.")

        for id_lain, kendaraan in data_kendaraan.items():
            if kendaraan["plat"] == plat and id_lain != id_kendaraan:
                raise ValueError("Nomor plat sudah digunakan.")

        pemilik = input("Nama pemilik baru: ").strip()
        jenis = input("Jenis baru (Mobil/Motor): ").strip().capitalize()
        merek = input("Merek baru: ").strip()
        tahun_input = input("Tahun baru: ").strip()

        if pemilik == "" or merek == "":
            raise ValueError("Data tidak boleh kosong.")

        if jenis not in ("Mobil", "Motor"):
```

```
            raise ValueError("Jenis kendaraan tidak valid.")

        if not validasi_tahun(tahun_input):
            raise ValueError("Tahun kendaraan tidak valid.")

        data.update({
            "plat": plat,
            "pemilik": pemilik,
            "jenis": jenis,
            "merek": merek,
            "tahun": int(tahun_input)
        })

        print("Data kendaraan berhasil diperbarui.")

    except ValueError as error:
        print("Error:", error)
```

4. Hapus Kendaraan (Delete)

Pengguna memasukkan ID kendaraan yang ingin dihapus. Sistem memeriksa hak akses dan meminta konfirmasi sebelum menghapus data.

Source Code:

```
def hapus_kendaraan(username, role):
    try:
        tampilkan_kendaraan()

        if not data_kendaraan:
            return

        print("\n========== HAPUS KENDARAAN ==========")
        id_input = input("Masukkan ID kendaraan: ").strip()

        if not id_input.isdigit():
            raise ValueError("ID harus berupa angka.")

        id_kendaraan = int(id_input)
        data = cari_kendaraan(id_kendaraan)

        if data is None:
            raise ValueError("Data tidak ditemukan.")

        if role != "Admin" and data["username"] != username:
            raise ValueError("Anda tidak memiliki akses.")
```

```
        print("Data yang akan dihapus:")
        tabel = PrettyTable()
        tabel.field_names = [
            "ID", "Nomor Plat", "Pemilik",
            "Jenis", "Merek", "Tahun"
        ]

        tabel.add_row([
            id_kendaraan,
            data["plat"],
            data["pemilik"],
            data["jenis"],
            data["merek"],
            data["tahun"]
        ])

        print(tabel)
        konfirmasi = input("Yakin hapus? (y/n): ").strip().lower()

        if konfirmasi == "y":
            del data_kendaraan[id_kendaraan]
            print("Data berhasil dihapus.")
        elif konfirmasi == "n":
            print("Penghapusan dibatalkan.")
        else:
            raise ValueError("Pilihan harus y atau n.")

    except ValueError as error:
        print("Error:", error)
```

F. Fitur Pengelolaan Akun oleh Admin

Fitur ini hanya tersedia untuk Admin dan mencakup tiga fungsi berikut.

1. Lihat Data Pengguna

Menampilkan daftar akun yang terdaftar, termasuk nomor urut, username, dan level akses.

Source Code:

```
def tampilkan_pengguna():
    print("========== DATA PENGGUNA ==========")

    tabel = PrettyTable()
```

```
    tabel.field_names = [
        "Nomor",
        "Username",
        "Role"
    ]

    if not akun:
        print("Belum ada data pengguna.")
        return

    for nomor, (username, data) in enumerate(akun.items(), start=1):
        tabel.add_row([
            nomor,
            username,
            data["role"]
        ])

    tabel.align = "l"
    print(tabel)
```

2. Edit Data Pengguna

Admin dapat mengubah username, password, dan level akses pengguna. Sistem memvalidasi username agar tidak duplikat serta membatasi pilihan role menjadi User atau Admin.

Source Code:

```
def edit_pengguna(username_login):
    try:
        tampilkan_pengguna()

        print("\n========== EDIT DATA PENGGUNA ==========")
        nomor = input("Pilih nomor akun yang ingin diedit: ").strip()

        if not nomor.isdigit():
            raise ValueError("Nomor harus berupa angka.")

        daftar_username = list(akun.keys())
        indeks = int(nomor) - 1

        if indeks < 0 or indeks >= len(daftar_username):
            raise ValueError("Akun tidak ditemukan.")

        username_lama = daftar_username[indeks]
```

```
        if username_lama == username_login:
            raise ValueError("Anda tidak dapat mengedit akun sendiri.")

        username_baru = input("Username baru: ").strip()
        password_baru = input("Password baru: ").strip()
        role_baru = input("Role baru (User/Admin): ").strip().capitalize()

        username_ada = False

        for i in akun:
            if i != username_lama and i == username_baru:
                username_ada = True

        if username_baru == "" or password_baru == "":
            raise ValueError("Username dan password wajib diisi.")
        elif username_ada:
            raise ValueError("Username sudah digunakan.")
        elif role_baru not in ("User", "Admin"):
            raise ValueError("Role harus User atau Admin.")

        akun[username_baru] = akun.pop(username_lama)
        akun[username_baru]["password"] = password_baru
        akun[username_baru]["role"] = role_baru

        for kendaraan in data_kendaraan.values():
            if kendaraan["username"] == username_lama:
                kendaraan["username"] = username_baru

        print("Data pengguna berhasil diperbarui.")

    except ValueError as error:
        print("Error:", error)
```

3. Hapus Akun Pengguna

Admin dapat menghapus akun pengguna dengan konfirmasi. Sistem mencegah penghapusan akun sendiri dan akun yang memiliki role Admin.

Source Code:

```
def hapus_pengguna(username_login):
    try:
        tampilkan_pengguna()
```

```
        print("\n========== HAPUS AKUN PENGGUNA ==========")
        nomor = input("Pilih nomor akun yang ingin dihapus: ").strip()

        if not nomor.isdigit():
            raise ValueError("Nomor harus berupa angka.")

        daftar_username = list(akun.keys())
        indeks = int(nomor) - 1

        if indeks < 0 or indeks >= len(daftar_username):
            raise ValueError("Akun tidak ditemukan.")

        target = daftar_username[indeks]

        if target == username_login:
            raise ValueError("Tidak dapat menghapus akun sendiri.")

        if akun[target]["role"] == "Admin":
            raise ValueError("Akun Admin tidak boleh dihapus.")

        konfirmasi = input(f"Yakin hapus akun {target}? (y/n):
").strip().lower()

        if konfirmasi == "y":
            del akun[target]
            print("Akun pengguna berhasil dihapus.")
        elif konfirmasi == "n":
            print("Penghapusan dibatalkan.")
        else:
            raise ValueError("Pilihan harus y atau n.")

    except ValueError as error:
        print("Error:", error)
```

G. Fitur Validasi Tahun dan Perhitungan Rekursif

Program menerapkan validasi tahun kendaraan untuk memastikan input berupa angka dan berada dalam rentang yang ditentukan. Selain itu, fungsi rekursif digunakan untuk menghitung jumlah kendaraan yang tersimpan.

Source Code:

```
def validasi_tahun(tahun):
    try:
```

```
        tahun = int(tahun)
        return 1899 < tahun <= datetime.now().year
    except (ValueError, TypeError):
        return False
```

Source Code perhitungan rekursif:

```
def hitung_kendaraan_rekursif(daftar_id, indeks=0):

    if indeks >= len(daftar_id):
        return 0

    return 1 + hitung_kendaraan_rekursif(daftar_id, indeks + 1)
```

H. Fitur Logout dan Keluar Program

Fitur logout mengakhiri sesi pengguna dan mengembalikan program ke menu utama. Sementara itu, menu keluar menghentikan program.

Source Code:

```
# logout
elif menu == "0":
    os.system("cls" if os.name == "nt" else "clear")
    print("Logout berhasil.")
    break

# keluar
elif opsi == "3":
    os.system("cls" if os.name == "nt" else "clear")
    print("Terima kasih telah menggunakan program.")
    break
```

4. Hasil Output

```
========== REGISTER ==========
Buat username: Robin
Buat password: 6767
Registrasi berhasil. Silakan login.

Tekan Enter untuk melanjutkan...
```

Gambar 4.1 Register

```
=== LOGIN ===
Masukkan username:
Masukkan password:
Username atau password salah! Sisa percobaan: 2
Masukkan username:
Masukkan password:
Username atau password salah! Sisa percobaan: 1
Masukkan username:
Masukkan password:
Username atau password salah 3 kali. Silahkan mulai ulang

Tekan Enter untuk melanjutkan...
```

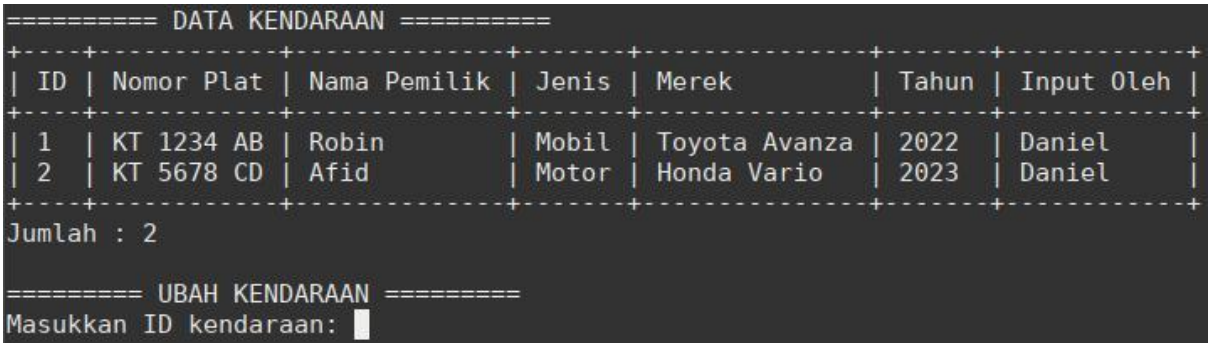
Gambar 4.2 Login

```
SISTEM PENDATAAN KENDARAAN .
1. Login
2. Register
3. Keluar
Pilih Menu : 3
Terima kasih telah menggunakan Program ini!
vario@vario:~/Downloads/Tugas Praktikum/Tugas APD/posttest/posttest-apd-5
$
```

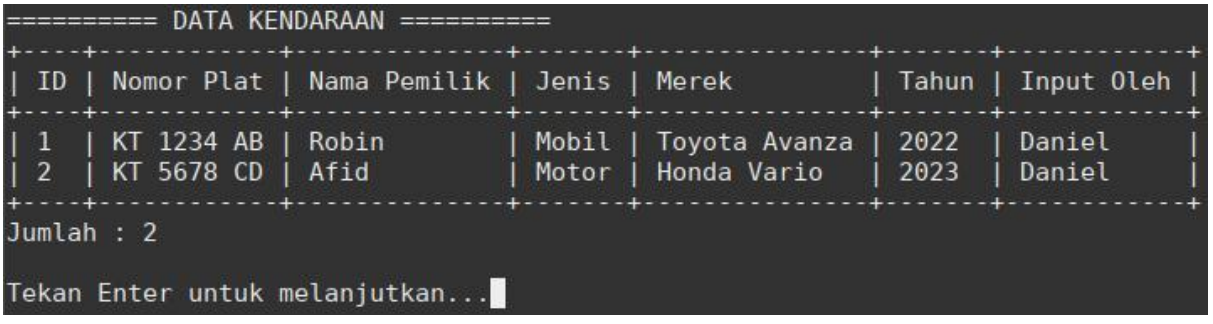
Gambar 4.3 Keluar



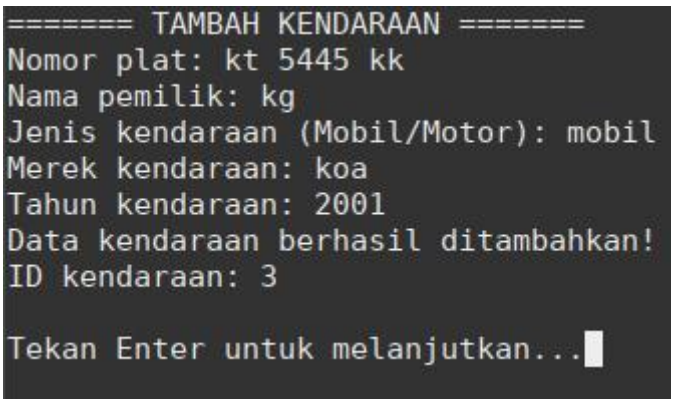
Gambar 4.4 Delete



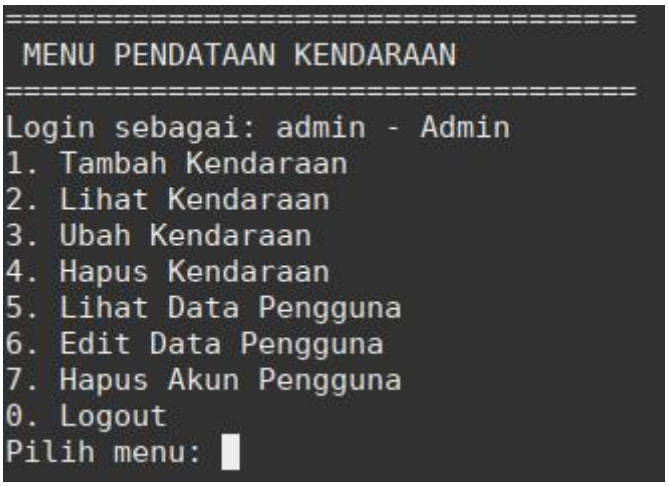
Gambar 4.5 Update



Gambar 4.6 Read



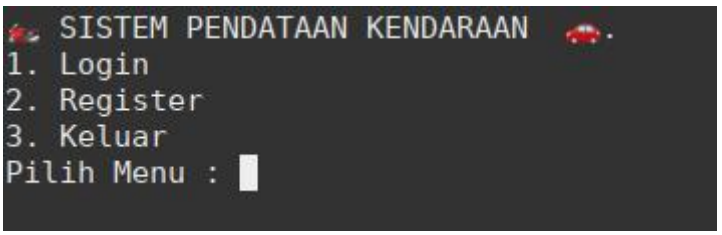
Gambar 4.7 Create



Gambar 4.8 Menu Admin



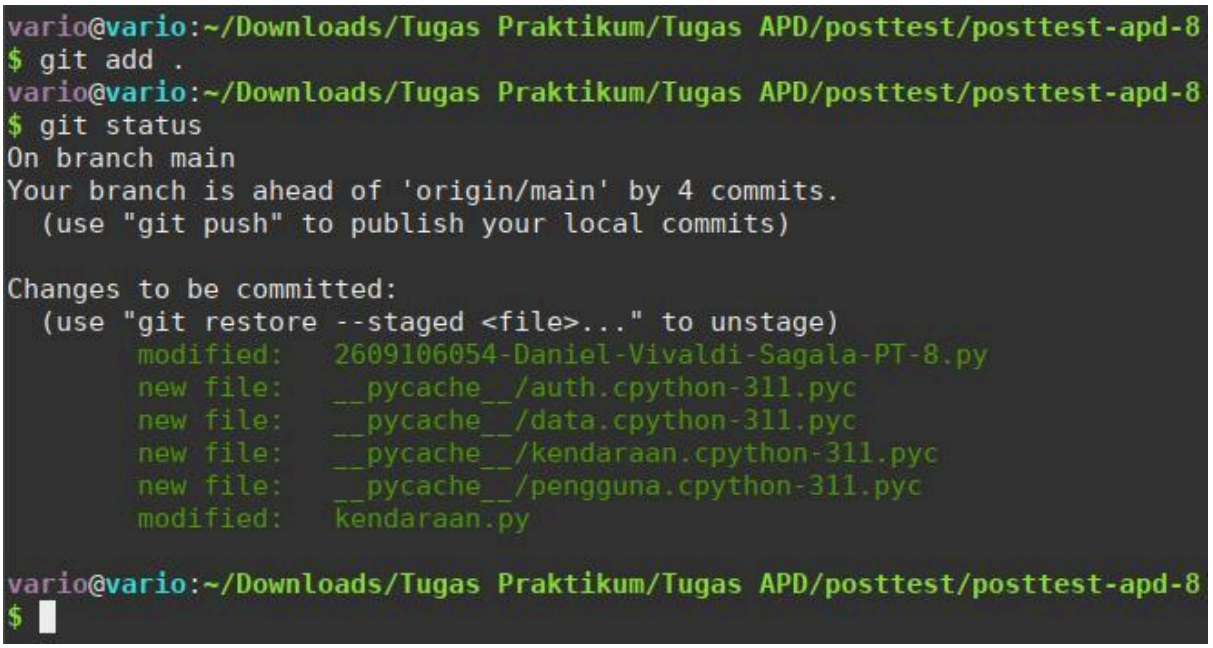
Gambar 4.9 Menu User



Gambar 4.10 Menu Utama

5. Langkah-langkah Git

5. 1 Git Add



Menambahkan file dengan cara "**git add nama_file**" atau jika kalian ingin menambahkan semua file kalian bisa menggunakan "**.**" Jadi "**git add .**" .lalu kalian bisa cek apakah file tersebut sudah terdaftar atau belum dengan cara mengetik "**git status**".

5.2 Git Commit

```
vario@vario:~/Downloads/Tugas Praktikum/Tugas APD/posttest/posttest-apd-8
$ git commit -m "file py"
[main 9be3c80] file py
 6 files changed, 15 insertions(+), 345 deletions(-)
 create mode 100644 posttest/posttest-apd-8/__pycache__/auth.cpython-311.pyc
 create mode 100644 posttest/posttest-apd-8/__pycache__/data.cpython-311.pyc
 create mode 100644 posttest/posttest-apd-8/__pycache__/kendaraan.cpython-311.py
c
 create mode 100644 posttest/posttest-apd-8/__pycache__/pengguna.cpython-311.pyc
vario@vario:~/Downloads/Tugas Praktikum/Tugas APD/posttest/posttest-apd-8
$
```

Mengetik **git commit -m "pesan yang ingin ditulis"** untuk melakukan commit atau konfirmasi perubahan yang terjadi pada repository kalian.

5.3 Git Push

```
vario@vario:~/Downloads/Tugas Praktikum/Tugas APD/posttest/posttest-apd-8
$ git push
Enter passphrase for key '/home/vario/.ssh/id_ed25519':
Enumerating objects: 36, done.
Counting objects: 100% (36/36), done.
Delta compression using up to 2 threads
Compressing objects: 100% (33/33), done.
Writing objects: 100% (34/34), 15.95 KiB | 1.59 MiB/s, done.
Total 34 (delta 12), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (12/12), completed with 1 local object.
remote: This repository moved. Please use the new location:
remote:   git@github.com:Ftyr3/praktikum-apd-2026.git
To github.com:Ftyr3/Praktikum_APD_B2-26.git
   0eb65b5..9be3c80  main -> main
vario@vario:~/Downloads/Tugas Praktikum/Tugas APD/posttest/posttest-apd-8
$
```

Melakukan upload file yang tadinya hanya berada pada komputer kalian ke Github dengan cara mengetik **"git push -u origin main"**. Jika berhasil maka outputnya sama seperti pada gambar diatas.